The Ultimate Greedy & Tricks Reference Guide

Two Pointers, Monotonic Stack, Medians, Sorting, Greedy

Sahil Raj (After Dark)

Contents

Chapter 1

Greedy Algorithms

Mental Model / Pattern

Core idea: Greedy algorithms are seductive — they *feel* right but frequently fail on subtle counter-cases. The three most reliable tools for greedy problems are: **(1) exchange arguments** (assume optimal differs from yours, swap to show it can't be better), **(2) sort and scan** (most greedy choices become obvious once elements are ordered by the right key), and **(3) fix the last operation first** (when operations commute or interact in a chain, reasoning about the final step collapses exponential branching into a linear scan).

1.1 Greedy with Wildcards: Forced Assignment + Uniqueness Check

Classic Problem

Recover an RBS (CF 2126B): A string of `(`, `)`, and `?` is given (guaranteed to have at least one valid RBS). Replace all `?`s to form an RBS. Determine if the valid replacement is **unique**.

When to use this pattern

When to use: You have wildcard characters (`?`) with a **fixed total count constraint** on how many must become each type. Before reaching for DP, always try **greedy filling**: the counts are forced, so there's essentially only one candidate assignment. Then check if it's truly unique.

When wildcards must satisfy a global count constraint (e.g., exactly $n/2$ opening and $n/2$ closing brackets), the assignment is nearly forced:

1. **Count fixed characters.** Compute $\text{need}_{(} = n/2 - \text{count}(\texttt{(})$ and $\text{need}_{)} = n/2 - \text{count}(\texttt{)})$. These quotas are rigid — no freedom.

2. **Greedily fill left-to-right.** Assign `(` from `?`s until the quota is met, then `)`.

3. **Validate with prefix balance.** A running balance (each `(` adds +1, each `)` subtracts 1) must never go negative.

4. **Check uniqueness with one swap.** The only dangerous swap is: the **last assigned** `(` swapped with the **first assigned** `)`. If this swapped string is also a valid RBS, the assignment is not unique (`No`); otherwise it is (`Yes`).

Common Trap / Gotcha

Common Trap: Do NOT reach for DP here. Tracking bracket balances across positions in a 2D table is a classic over-engineering trap. The counts are **forced**, so there is only one candidate. The only question is uniqueness, which reduces to a single swap check.

Complexity

$\mathcal{O}(n)$ per test case: one fill pass, one validation pass, two linear scans for the swap indices.

1.2 Suffix Minimum Modification (Gravity Shift)

Classic Problem

It All Went Sideways (CF 2227E): n columns of cubes; gravity shifts right. Cubes slide horizontally as far as possible. Before the shift, you can remove **at most one** cube. Maximize the number of cubes that move.

When to use this pattern

When to use: A physical/simulation problem where elements "fall" or "slide" in one direction, and you want to maximize disruption with a single small modification. Build a **suffix minimum array** and look for the longest run of equal values.

The key observation is: after the gravity shift, a cube at height h in column i stays in place *only if* all columns to its right also have height $\geq h$. Therefore:

- The number of cubes that **stay** in column i is exactly $\text{suf}[i] = \min_{j \geq i} a_j$.
- The base number of **moving** cubes is $\sum_i (a_i - \text{suf}[i])$.

Removing one cube from column k can decrease $\text{suf}[i]$ by 1 for a contiguous segment of columns where the suffix minimum is identical. If this block has length L, the net gain is $\Delta = L - 1$ (we removed one cube but liberated L static cubes).

Therefore: find the longest contiguous block of equal values in suf, and add $L_{\max} - 1$ to the base answer.

Optimization Note

Why $L - 1$ and not L? You removed one cube from the system entirely (that cube itself is gone), so the gain is the block length L minus the 1 you sacrificed.

Complexity

$\mathcal{O}(n)$ per test case: one right-to-left sweep for suffix min, two left-to-right sweeps for base count and max block length.

1.3 Greedy Casework on Graph Components (Clique Detection)

Classic Problem

Make It Connected (CF 1761E): Given an undirected graph, in one operation pick a vertex u and invert all edges incident to u (add missing, remove existing). Find the minimum operations to make the graph connected.

When to use this pattern

When to use: A graph modification problem where each operation affects a single vertex's neighbourhood. Identify the **structural type** of each connected component (isolated? non-clique? perfect clique?) and apply greedy casework — from cheapest to most expensive.

The key is recognizing what each operation does to a component, then greedily applying the cheapest fix first:

1. **Case 0 (already connected):** 0 operations.
2. **Case 1 (isolated vertex exists):** Pick the isolated vertex. It connects to *everyone*. 1 operation.
3. **Case 2 (a non-clique component exists):** A non-clique has an internal missing edge (u, v). Applying the operation to the **minimum-degree** vertex u in such a component:
   - Connects u to all other components (merging them).
   - Connects u to its non-neighbor v within the component.
   - v acts as an anchor, keeping u inside its original component.

   1 operation.
4. **Case 3 (all cliques, > 2 components):** Pick one vertex from C_1 and one from C_2; they cross-link all cliques. 2 operations.
5. **Case 4 (exactly 2 cliques):** Invert the entire smaller clique — all its vertices disconnect from each other but fully connect to the larger clique. $\min(|C_1|, |C_2|)$ operations.

Common Trap / Gotcha

Checking if a component is a clique: A component of size S is a clique if and only if every vertex inside it has degree exactly $S - 1$ (within the component). If any vertex has lower degree, it's **not** a clique.

Complexity

$\mathcal{O}(n^2)$ per test case. Reading the adjacency matrix and DFS to find components: proportional to n^2 possible edges. With $\sum n \leq 4000$, $n^2 \leq 1.6 \times 10^7$.

1.4 Exchange Argument: Proving Greedy Optimality

When to use this pattern

When to use: You have a greedy idea but can't prove it. Use the **exchange argument**: assume an optimal solution exists that does NOT make your greedy choice. Show you can swap elements to match the greedy choice without worsening the outcome.

The exchange argument is the most rigorous way to prove a greedy strategy. The general template:

1. **Assume** an optimal solution OPT that differs from the greedy solution G at some step.
2. **Identify** the first index/choice where they differ.
3. **Swap** the greedy choice into OPT at that position, adjusting the rest of OPT accordingly.
4. **Show** that the swapped solution is at least as good as (or equal to) OPT.
5. **Conclude** that the greedy choice never hurts, so G is optimal.

Common greedy orderings that exchange arguments validate:

- **Interval Scheduling:** Sort by end time. Swapping a later-ending interval in OPT for the greedy (earlier-ending) one never reduces the number of compatible intervals.
- **Huffman / Task Scheduling:** Sort tasks by a ratio (e.g., deadline/weight). Swapping two adjacent out-of-order tasks strictly improves the total weighted lateness.
- **Coin / Change Problems:** Take the largest denomination first. Swapping a smaller-denomination choice for a larger one uses fewer coins.

Common Trap / Gotcha

Danger: Always try to break your greedy with small adversarial test cases BEFORE attempting the exchange argument. If you can find a counterexample, you've saved yourself from writing a wrong proof.

1.5 Greedy on Sorted Arrays: Sort + Scan

When to use this pattern

When to use: The optimal greedy choice at each step depends only on the **relative ordering** of elements. If you're stuck, ask: "what if I sort by start time? End time? Difference? Ratio?" One of these orderings will usually collapse the problem.

Most greedy algorithms operate on a sorted array. Common patterns:

- **Activity Selection / Interval Scheduling:** Sort intervals by **end time**. Greedily pick the interval that ends earliest and doesn't conflict with the last chosen.
- **Minimize Sum of Products:** To minimize $\sum a_i \cdot b_i$, sort a ascending and b descending (pair the largest a_i with the smallest b_i). This follows from the rearrangement inequality.
- **Minimize Maximum Discrepancy:** Sort the array. The optimal sequence always grows as a **contiguous window** in the sorted array (interval DP applies once you've sorted).
- **Fractional Knapsack:** Sort by value/weight ratio descending. Greedily take the most efficient items.
- **Task Scheduling with Deadlines:** Sort tasks by deadline. Use a greedy scheduler (or a priority queue) to always take the most profitable task that fits before its deadline.

Optimization Note

Sort-and-scan template:

1. Sort by the key that makes the greedy choice locally obvious.
2. Scan left to right, making the locally optimal decision at each step.
3. Maintain only the state needed for the current decision (often just a running count, sum, or a small priority queue).

Chapter 2

Median Tricks

Mental Model / Pattern

Core idea: The median minimizes the sum of absolute differences. If you need to find a point x that minimizes $\sum |A[i] - x|$, x must be the median of array A. For dynamic medians, maintaining two heaps (a max-heap for the lower half and a min-heap for the upper half) allows $\mathcal{O}(\log n)$ updates and $\mathcal{O}(1)$ median queries.

2.1 Minimizing Absolute Distances

2.2 Dynamic Median (Data Stream)

2.3 Sliding Window Median

Optimization Note

Quick Select for Median: If you only need to find the median (or k-th element) of an unsorted array *once*, use `std::nth_element` which runs in $\mathcal{O}(n)$ average time instead of sorting $\mathcal{O}(n \log n)$.

Complexity

Data stream median: $\mathcal{O}(\log n)$ per insertion, $\mathcal{O}(1)$ query. Sliding window median: $\mathcal{O}(\log k)$ per slide using balanced BST (multiset).

Common Trap / Gotcha

Even length median: When k or N is even, the median is the average of the two middle elements. Beware of integer overflow when adding two large integers before dividing by 2! Use `((double)a + b) / 2.0` or `a / 2.0 + b / 2.0`.

Chapter 3

Sorting Tricks

Mental Model / Pattern

Core idea: Sorting is not just a preprocessing step — it is often the **key insight** that transforms an $\mathcal{O}(n^2)$ brute-force into an $\mathcal{O}(n \log n)$ elegant scan. The right sort key collapses the search space: once elements are ordered correctly, the greedy or DP choice becomes local and obvious.

3.1 Sort + Scan (Greedy after Sorting)

When to use this pattern

When to use: You need to make a sequence of locally optimal choices. The correct choice at each step depends on the relative order of elements. Try sorting by: end time, start time, ratio (value/weight), difference, or absolute value.

After sorting, a single left-to-right scan handles most greedy problems:

- **Activity Selection:** Sort by end time. Take the earliest-ending non-conflicting interval.
- **Minimize Sum of Products (Rearrangement Inequality):** Sort a ascending and b descending to minimize $\sum a_i b_i$. Sort both in the same direction to maximize it.
- **Fractional Knapsack:** Sort items by value/weight ratio descending. Greedily take as much of the best item as possible.

3.2 Sort + Interval DP (Minimize Discrepancy)

Classic Problem

Sports Festival (CF 1509C): Given n running speeds, arrange them in any order to minimize the sum of discrepancies $d_i = \max(\text{first } i) - \min(\text{first } i)$.

When to use this pattern

When to use: You are choosing elements one by one and care about the running min/max (i.e., the range). Sort the array first — then the chosen prefix always forms a **contiguous window** in sorted order.

Key Observation: In an optimal ordering, the set of chosen elements is always a contiguous interval $[i, j]$ of the sorted array. Why? Any element between the current min and max costs 0 additional discrepancy, so

there's never a reason to leave a "gap".

DP State: Let $dp[i][j]$ = minimum total discrepancy to arrive at the sorted interval $[i, j]$.

Transition: To reach $[i, j]$, the previous step was either $[i+1, j]$ (we just added the new minimum v_i) or $[i, j-1]$ (we just added the new maximum v_j). In either case, the current discrepancy added is $v_j - v_i$:

$$dp[i][j] = (v_j - v_i) + \min(dp[i+1][j],\ dp[i][j-1])$$

Base: $dp[i][i] = 0$. **Answer:** $dp[0][n-1]$.

Complexity

$\mathcal{O}(n \log n)$ to sort + $\mathcal{O}(n^2)$ DP. With $n \leq 2000$: 4×10^6 transitions, comfortably within limits.

3.3 Harmonic Series Trick (Sum over Divisors)

When to use this pattern

When to use: You iterate over all possible values of a divisor or a block size x from 2 to M, and for each x you process M/x blocks. The total work is $\sum_{x=2}^{M} M/x \approx M \ln M = \mathcal{O}(M \log M)$.

A classic example: for each divisor x, group array elements into blocks of size x and process each block in $\mathcal{O}(1)$ using prefix sums. The nested loop:

```
for x = 2 to M:
    for k = 1 to ceil(M/x):
        process block [(k-1)*x+1, k*x]
```

runs in $\mathcal{O}(M \log M)$ total due to the harmonic series bound $\sum_{x=1}^{M} \lfloor M/x \rfloor \approx M \ln M$.

Optimization Note

Precompute frequency + prefix sums first. Store $cnt[v]$ = number of elements with value v, and $pre[i] = \sum_{j=1}^{i} cnt[j]$. Then any range frequency query is $\mathcal{O}(1)$.

3.4 Counting Inversions (Merge Sort / BIT)

When to use this pattern

When to use: You need to count pairs (i, j) with $i < j$ and $a_i > a_j$. Also applies when counting pairs satisfying a threshold inequality on sorted data.

Two standard approaches:

- **Merge Sort:** During the merge step, count elements from the right half that are placed before elements from the left half. $\mathcal{O}(n \log n)$.
- **BIT (Fenwick Tree):** Process elements right-to-left. For each a_i, query the BIT for the count of elements $< a_i$ already seen. Update the BIT at position a_i. $\mathcal{O}(n \log n)$.

Common Trap / Gotcha

Duplicate elements: Be precise about strict vs. non-strict inequalities when counting. For pairs with equal values, decide upfront whether they count as inversions.

Chapter 4

Two Pointers

Mental Model / Pattern

Core idea: Two pointers work when the problem has a **monotonic shrink/expand property**: moving one pointer in one direction while moving the other in the same or opposite direction always monotonically changes some value of interest. The key insight is that you never need to revisit a position, giving $\mathcal{O}(n)$ instead of $\mathcal{O}(n^2)$.

4.1 Classic Two Pointers (Opposite Ends)

When to use this pattern

When to use: You have a sorted array and need to find pairs (i, j) with $i < j$ satisfying some condition (e.g., $a_i + a_j = T$, or $a_j - a_i \leq K$).

Set $i = 0$, $j = n - 1$. At each step, compare $a_i + a_j$ with the target:

- If sum $<$ target: increment i (need a larger element from the left).
- If sum $>$ target: decrement j (need a smaller element from the right).
- If sum $=$ target: record and move both pointers.

Complexity

$\mathcal{O}(n \log n)$ for sorting + $\mathcal{O}(n)$ for the two-pointer scan.

4.2 Sliding Window (Same-Direction Two Pointers)

When to use this pattern

When to use: Find the smallest/largest subarray satisfying a property (e.g., sum $\geq K$, at most k distinct elements). The window can expand by moving the right pointer and shrink by moving the left pointer. The critical requirement: the property must be **monotonic** as the window grows or shrinks.

Template for "smallest subarray with sum $\geq K$":

1. Maintain a running window sum.
2. Expand right: add $a[r]$ to the sum.

3. Shrink left: while sum $\geq K$, record the window length and remove $a[l]$, then increment l.
4. $\mathcal{O}(n)$ since each pointer moves at most n times.

Optimization Note

Sliding Window Maximum (Monotonic Deque): To find $\max(a[l..r])$ for a fixed window of size k in $\mathcal{O}(1)$ per slide, maintain a deque of indices in **decreasing order of value**. The front is always the current maximum. Pop from the front when it's out of the window; pop from the back when a new element is larger.

4.3 Two Pointers on Sorted Events (Merge-style)

When to use this pattern

When to use: You have two sorted arrays A and B, and for each element in A you need to find how many elements in B satisfy an inequality. Use a single left pointer on B that only moves right as you process A left-to-right.

This is the merge step in merge sort, and also the backbone of many counting problems:

1. Sort A and B.
2. Walk A left-to-right; maintain a pointer j on B.
3. For each a_i, advance j while $B[j] \leq f(a_i)$. The count of valid B elements is j.
4. Total work: $\mathcal{O}(n \log n)$ for sorting + $\mathcal{O}(n)$ for the scan.

4.4 Prefix Sums + Binary Search (Alternative to Two Pointers)

When to use this pattern

When to use: The window isn't strictly contiguous, or you can't guarantee the monotonic shrink property. Build a prefix sum array and use `std::lower_bound` to find boundaries in $\mathcal{O}(\log n)$ per query.

Common Trap / Gotcha

Negative numbers break the monotonicity! Classic two pointers on subarray sums assume all elements are positive. If there are negatives, the window sum is not monotone as you expand/shrink, and two pointers may miss valid windows. Fall back to prefix sums + monotonic deque or segment trees.

Chapter 5

Monotonic Stack & Queue

Mental Model / Pattern

Core idea: A monotonic stack maintains elements in a strictly increasing or decreasing order. Whenever a new element violates the order, you **pop** until the order is restored. Each element is pushed and popped at most once, giving $\mathcal{O}(n)$ total. The popped elements reveal a key relationship: the element being pushed is the first element to the right that is larger (or smaller) than the popped one.

5.1 Next Greater / Previous Greater Element

When to use this pattern

When to use: For each element a_i, find the nearest element to its left (or right) that is strictly greater (or smaller). This is the foundation of dozens of harder problems.

Next Greater Element (right): Iterate left-to-right. Maintain a decreasing stack. When a_i pops element a_j from the stack, a_i is the **next greater element** for a_j.

Previous Greater Element (left): At index i, after popping all $\leq a_i$, the top of the stack is the **previous greater element** for a_i.

Complexity

$\mathcal{O}(n)$ amortized: each element pushed and popped at most once.

5.2 Largest Rectangle in Histogram

Classic Problem

Classic Problem: Given an array of bar heights, find the largest rectangular area in the histogram.

When to use this pattern

When to use: Any problem that asks for the largest "box" that fits under a profile of heights. Also arises in 2D variants: for a binary matrix, convert each row to a histogram and apply this trick per row.

For each bar i, find:

- L_i: index of the nearest bar to the left that is **strictly shorter**.

- R_i: index of the nearest bar to the right that is **strictly shorter**.

The largest rectangle with bar i as the limiting height has area $h_i \times (R_i - L_i - 1)$.

Both L and R are computed simultaneously in one pass using a monotonic stack: maintain an **increasing** stack. When h_i pops h_j, i is R_j (right boundary); the new stack top after popping is L_j (left boundary).

5.3 Maximum GEQ Sum (Fix Max, Query Subarray Sum)

Classic Problem

Max GEQ Sum (CF 1359E): Check whether $\max(a_i, \ldots, a_j) \geq a_i + \cdots + a_j$ holds for **all** pairs (i, j).

When to use this pattern

When to use: A condition involves both the maximum and the sum of the same subarray. Fix the maximum element first using a monotonic stack to find its domain, then query the max-subarray-sum within that domain.

Strategy: Fix the maximum, not the subarray.

For each element $a[k]$, find the maximal range $[L_k, R_k]$ where $a[k]$ is the **strict maximum**:

- L_k = nearest index with a strictly greater element to the left.
- R_k = nearest index with a strictly greater element to the right.

These boundaries are found using monotonic stacks in $\mathcal{O}(n)$.

Then, for each k, query: "is the maximum subarray sum inside $[L_k + 1, R_k - 1]$ greater than $a[k]$?"

This query is answered by a **segment tree for max-subarray-sum**. Each node stores four fields:

- **sum**: total sum of the range.
- **pre**: max prefix sum.
- **suff**: max suffix sum.
- **ans**: max subarray sum anywhere in the range.

Merge rule: $p.\text{ans} = \max(\ell.\text{ans},\ r.\text{ans},\ \ell.\text{suff} + r.\text{pre})$.

Common Trap / Gotcha

Handling duplicates: When using `<=` in the monotonic stack (pop elements $\leq$ current), the **rightmost** copy of a duplicated value owns the range. Group equal elements by value using a map to avoid double-counting.

Complexity

$\mathcal{O}(n \log n)$: $\mathcal{O}(n)$ for monotonic stacks + $\mathcal{O}(n \log n)$ to build and query the segment tree.

5.4 Monotonic Queue (Sliding Window Min/Max)

When to use this pattern

When to use: You need the minimum (or maximum) of a sliding window of fixed size k efficiently. Use a **deque** (double-ended queue) of indices, maintained in increasing (for min) or decreasing (for max) order of values.

Sliding window minimum:

1. Push index i to the back of the deque (after popping from the back while `a[deque.back()] >= a[i]`).
2. Pop from the front while `deque.front() <= i - k` (out of window).
3. `a[deque.front()]` is the window minimum.

Each index is pushed and popped at most once: $\mathcal{O}(n)$ total.

Optimization Note

Connection to DP optimization: Monotonic queue optimizes DP transitions of the form $dp[i] = \min_{j \in [i-k, i-1]} dp[j] + f(i)$. The queue maintains the minimum $dp[j]$ over the valid window of j values, reducing the DP from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$.

Chapter 6

Subarray Sum Tricks

Mental Model / Pattern

Core idea: Arrays containing only ± 1 have the **Intermediate Value Property** for subarray sums. Because adding or removing an element changes the sum by at most ± 1, a subarray of ± 1 can form *every single integer* between its minimum possible sum and its maximum possible sum.

6.1 The ± 1 Array with an Outlier (x)

Classic Problem

The Setup: You have an array A filled with 1s and -1s, except for exactly one element which has been changed to a large value x. Find all possible sums that can be formed by choosing a contiguous subarray.

Optimization Note

Why does this work? Any subarray that does *not* include x consists solely of 1s and -1s, so its sums smoothly cover the range $[L_1, R_1]$. Any subarray that *does* include x essentially has a fixed contribution x, plus some ± 1s on the left and right, so its sums smoothly cover $[L_2, R_2]$.

Chapter 7

Advanced CP Tricks & Invariants

Mental Model / Pattern

Core idea: Some problems completely break down if you view them from the forward simulation perspective. Shifting your perspective — executing backwards, binarizing arrays, applying difference constraints, or finding modulo invariants — turns $\mathcal{O}(n^2)$ headaches into $\mathcal{O}(n)$ poetry.

7.1 Difference Constraints on 1D Arrays

Classic Problem

Maximum Prefix Sums (CF 2231D): You need to reconstruct an array given partial information about its elements and the maximums of its prefix sums.

When to use this pattern

When to use: You are asked to reconstruct an array or find valid bounds, and you are given equality or inequality constraints between adjacent elements or prefix sums. If the constraints form a 1D path, you can avoid heavy graph algorithms and just sweep left and right.

Instead of trying to guess missing elements directly, try to reconstruct the **prefix sums** array b. When you have equality constraints between adjacent elements like $b_i - b_{i-1} = a_i$, you can split them into two inequalities:

$$b_{i-1} \leq b_i - a_i \quad \text{and} \quad b_i \leq b_{i-1} + a_i$$

This is a **System of Difference Constraints**. Usually, this requires Bellman-Ford $\mathcal{O}(VE)$ to find the tightest upper bounds. However, because the graph is just a 1D path, you can satisfy all constraints with just **two linear sweeps**:

- **Backward Pass** (Right to Left): Tighten $b_{i-1} = \min(b_{i-1}, b_i - a_i)$.
- **Forward Pass** (Left to Right): Tighten $b_i = \min(b_i, b_{i-1} + a_i)$.

This optimally propagates all upper bounds in $\mathcal{O}(n)$.

7.2 The Binarization Median Trick

Classic Problem

Me When Median Problem (CF 2229D): Extracting medians from sliding subsets and replacing elements, trying to maximize the final remaining element.

When to use this pattern

When to use: The problem asks you to **maximize/minimize a median**, find the "median of medians", or repeatedly extract the middle element from subsets/sliding windows.

Whenever a problem asks you to **maximize a median** or deal with sorting small subsets to extract middle elements, always think of **Binary Search on the Answer**.

1. **Guess a target value x.**
2. **Binarize the array:** If an element is $\geq x$, map it to $+1$. If it's $< x$, map it to -1.
3. **Analyze the battle:** The problem simplifies dramatically. A pair of $+1$s becomes a "strong win" that can infect adjacent blocks. You just need to check if the total number of $+1$ groups can overpower the -1 groups. By completely ignoring the actual array values and only caring about the relative ± 1 states, a complex multiset simulation turns into a simple $\mathcal{O}(n)$ sweep!

7.3 Greedy Reverse Execution

Classic Problem

We Be Flipping (CF 2229C2): You can multiply a prefix of length i by -1 if $a_i > 0$. Maximize the total sum.

When to use this pattern

When to use: You are given a sequence of commutative operations (like flipping signs or adding values to prefixes/suffixes) and forward simulation causes exponential branching. Ask yourself: "What happens if I fix the **last** operation first?"

Sometimes, simulating operations forward requires a complex DP. Instead, **think about the final operation**. Because operations commute, what happens if we execute the operation at index idx **last**?

- It flips the entire prefix up to idx.
- For the elements before idx to end up **positive** (to maximize sum), they must be **negative** right before this final operation!

You can perfectly force all elements before idx to be negative by simply walking backwards from $idx - 1$ down to 1 and flipping any positive element you see. This guarantees that picking idx as the "maximum operation index" will magically turn **every single element** before it into its absolute value! Thus, you just iterate over all possible maximum indices and calculate the answer in $\mathcal{O}(1)$ using prefix sums.

7.4 Inverse Permutations & LCP

Classic Problem

Fixed Prefix Permutations (CF 1792D): Given n permutations, find the maximum "beauty" of $p \cdot q$, which is the longest prefix of $1, 2, \ldots, k$.

When to use this pattern

When to use: You are dealing with the **product of permutations** $(p \cdot q)$, or conditions involving the values matching their 1-based indices (identity prefixes). If the problem asks "at what index is value x?", you need the inverse permutation.

To maximize the identity prefix of $r = p \cdot q$, we need $q_{p_x} = x$. In terms of inverse permutations, this mathematically translates to:

$$q_x^{-1} = p_x$$

Thus, the "beauty" of $p \cdot q$ is exactly the **Longest Common Prefix (LCP)** between p and q^{-1}. Instead of building a Trie, you can solve this by:

1. Computing the inverse of every permutation.
2. Sorting the array of inverses lexicographically.
3. For each p, use `std::lower_bound` on the sorted inverses array. The longest common prefix is guaranteed to be found at the insertion point or its immediate neighbor!

7.5 The Modulo Invariant

Classic Problem

Fullmetal Bitchemist (CF 2237D): Replace adjacent `00` with `1`, and `11` with `0`. Count substrings that can be reduced to length 1.

When to use this pattern

When to use: You have a string or array reduction problem where adjacent elements merge or change state (e.g., `00` $\to$ `1`). If simulating the merges is too slow, map the states to numbers and look for a value that remains constant modulo k.

Assign numerical values to the binary characters: `0` $\to 1$ and `1` $\to 2$. Let's look at the sum under operations:

- `00` $\to$ `1`: $1 + 1 = 2 \implies 2$.
- `11` $\to$ `0`: $2 + 2 = 4 \implies 1$.

Notice that $4 \equiv 1 \pmod 3$, and $2 \equiv 2 \pmod 3$. **The sum of the string modulo 3 never changes!** Since a fully reduced string of length 1 must be either '0' (sum 1) or '1' (sum 2), any valid substring **must have an initial sum** $\not\equiv 0 \pmod 3$. This beautiful invariant transforms a complex string reduction problem into a simple prefix-sum counting problem (subtracting bad substrings with sum $\equiv 0 \pmod 3$, and un-reducible alternating odd-length strings).